

Talos

Implementation Plan for an Agent-Agnostic Coding Orchestration Platform

Prepared for Paul Bernard | 2026-07-09

Purpose

This document describes how to build a self-hosted, visual AI agent harness for managing coding work across many repositories, projects, portfolios, and agent providers. The platform does not build a new coding agent. It orchestrates existing agents such as Claude Code, OpenAI Codex, OpenCode, OpenHands, Gemini CLI, and similar tools through adapters, command-line harnesses, APIs, or protocols such as ACP/MCP where available.

Table of Contents

- 1. Executive summary
- 2. How this compares to Hermes, OpenHands, Vibe Kanban, and Agent OS
- 3. Product definition and non-goals
- 4. Target architecture
- 5. Monorepo and deployment model
- 6. Core services
- 7. Agent adapter layer
- 8. Workspace and execution model
- 9. Data model
- 10. API surface
- 11. Event and queue design
- 12. Security and governance
- 13. Subscription vs API-key execution strategy
- 14. Project configuration spec
- 15. Dashboard and UX scope
- 16. Implementation phases and tickets
- 17. Testing strategy
- 18. Dokploy deployment plan
- 19. Risks and mitigations
- 20. Claude Code execution prompt
- 21. References

1. Executive summary

Talos is a web-based control plane that manages projects, tasks, agent runs, workspaces, reviews, approvals, tests, pull requests, and deployment triggers. It should not attempt to build a new coding agent from scratch. The platform should treat coding agents as replaceable execution engines behind a stable adapter interface.

Core decision

Use a single monorepo, but deploy multiple services. This gives you one codebase for development and versioning, while keeping the web UI, API, orchestrator, worker runners, and integrations independently deployable in Dokploy.

Canonical product naming

The platform name is Talos. Use Talos from the first commit across repository names, service names, package names, environment variables, Docker services, documentation titles, and default UI labels. Avoid previous working-title names in generated code.

Codebase surface	Canonical Talos naming
Repository/root folder	talos/
Product/UI name	Talos
Java base package	io.talos
Docker service prefix	talos-api, talos-web, talos-orchestrator, talos-runner-supervisor
Environment variable prefix	TALOS_
Database name	talos
Queue exchange/routing prefix	talos.events / talos.task.run.requested
Project config file	talos.yaml; optional legacy aliases can be added later but must not be the MVP default

The minimum useful product should support these actions:

- Register a repository and stack profile.
- Create a task from a dashboard or remote trigger.
- Choose or automatically route to an agent.
- Create an isolated git worktree or clone for the run.
- Launch the selected agent through an adapter.
- Stream logs and capture artifacts.
- Run build/test commands defined per project.
- Show a review page with diff, test results, risk summary, and approval actions.
- Open a pull request or trigger Dokploy only after approval.

2. How this compares to Hermes, OpenHands, Vibe Kanban, and Agent OS

Your architecture is similar in spirit to existing tools, but it is narrower and more operator-focused. The goal is not a general personal assistant and not just a coding UI. It is a governance and orchestration layer for multiple existing coding agents across multiple projects.

Tool / Pattern	What it already does	How your platform differs
OpenHands / Agent Canvas	Self-hostable coding-agent platform with browser UI, SDK, APIs, model-agnostic positioning, ACP support, and Docker sandbox patterns for isolated execution.	Use similar sandbox/runtime ideas, but build a stronger project dashboard, Kanban board, Dokploy deployment integration, and your own provider-agnostic adapter layer. OpenHands can be one adapter/runtime, not necessarily the entire product.
Hermes / OpenClaw style assistants	Multi-channel assistant/gateway style systems with skills, memories, MCP/ACP bridges, and chat-based triggering.	Borrow the remote-trigger and skills concept, but make the center of gravity software engineering: projects, branches, tests, PRs, approvals, and deployments.
Vibe Kanban	Kanban issues, agent workspaces, branches, terminals, dev servers, diff review, and app previews for coding agents.	It is closest to the Kanban idea, but your version should be self-hosted around your stack, with formal project registry, policy engine, Dokploy integration, and subscription/API execution strategy.
Nimbalyst	Visual local workspace for Codex, Claude Code, OpenCode, sessions, worktrees, visual editing, approvals, and task management.	Useful UX inspiration, but your target is a remote web control plane hosted on your VPS, not only a local visual workspace.
BuilderMethods Agent OS	Captures coding standards and instructions for AI-powered development.	Treat this as a documentation/standards idea, not a full runtime. Your platform includes standards, but also execution, orchestration, review, and deployment.
GitHub Agent HQ / Copilot agents	GitHub-native mission control for multiple agents and pull-request workflows.	Good competitor/reference point, but less self-hosted and less customizable around your VPS, Dokploy, WhatsApp/Telegram, and local/subscription CLI modes.

Conclusion

Yes, the architecture with separate front end, back end, database, queue, cache, orchestrator, and agent runners is consistent with how serious agent platforms are evolving. The difference is that your product should be built as a monorepo of services, not one giant runtime. The agent runners must remain isolated because they execute untrusted or semi-trusted code and shell commands.

3. Product definition and non-goals

3.1 Product definition

Talos is a self-hosted, agent-agnostic development control plane for orchestrating coding agents across a portfolio of projects. It gives the user a central dashboard for project status, task planning, agent execution, review gates, and deployment triggers.

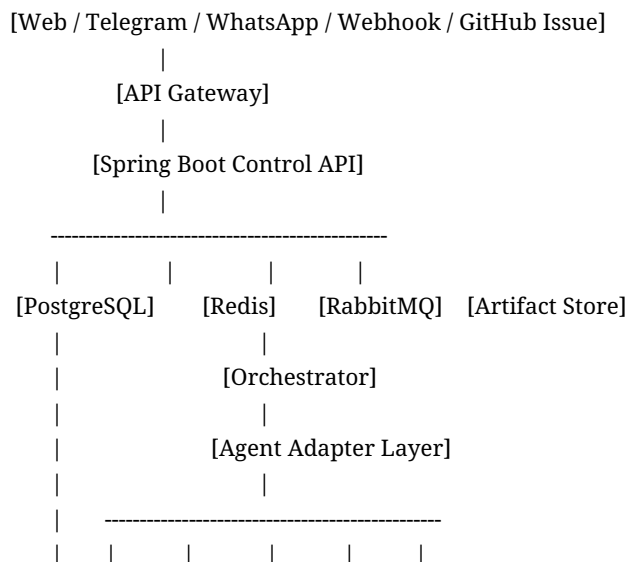
3.2 Primary users

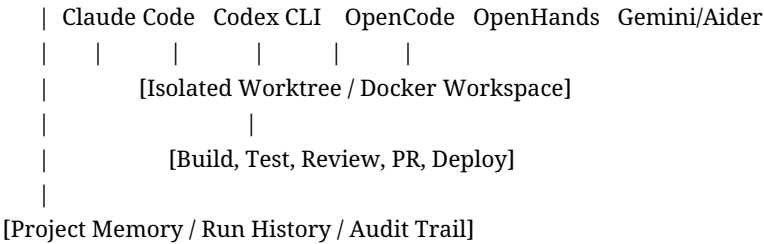
- Solo technical founder managing several products.
- Freelancer managing multiple client repositories.
- Small dev agency using Claude Code, Codex, OpenCode, or other coding agents.
- Internal DevOps/IT team that wants AI coding automation with governance.

3.3 Non-goals

- Do not build a new LLM or coding model.
- Do not start by building a full IDE.
- Do not let agents work directly on production branches.
- Do not allow unattended production deployment in the MVP.
- Do not rely on subscription credentials as a SaaS business model unless the execution happens on the customer's own machine and complies with provider terms.

4. Target architecture





4.1 Service boundaries

Service	Recommended technology	Responsibility
web	Angular	Dashboard, Kanban, project registry, run viewer, diff/review UI, approval actions.
api	Spring Boot	Auth, REST APIs, task/project persistence, approval workflow, integration registry, audit trail.
orchestrator	Python	Job dispatch, adapter routing, run state transitions, policy checks, step orchestration.
runner supervisor	Python or Go	Starts per-task containers, manages git worktrees, streams logs, collects diffs/artifacts.
worker images	Docker	Stack-specific execution environments: Java, Node, Python/Odoo, Flutter, generic agent CLI.
postgres	PostgreSQL + pgvector later	System of record for users, projects, tasks, runs, approvals, memory chunks.
rabbitmq	RabbitMQ	Durable async job and event queue.
redis	Redis	Short-lived run state, locks, streams, rate limits, websocket fan-out state.
artifact storage	Local volume first; MinIO/S3 later	Logs, patches, screenshots, build reports, generated docs.

5. Monorepo and deployment model

Build the platform as one monorepo with separate apps and deployable services. This is different from a single-process app. The monorepo keeps shared contracts and developer workflow simple, while service boundaries keep execution safe and scalable.

```
talos/
  apps/
    web/           # Angular dashboard
    api/          # Spring Boot API
    orchestrator/  # Python orchestration service
```

```
runner-supervisor/    # Workspace/container manager
telegram-adapter/     # Optional Phase 3
whatsapp-adapter/     # Optional Phase 4
```

```
workers/
base-agent-runner/
java-runner/
node-runner/
python-runner/
odoo-runner/
flutter-runner/
```

```
packages/
contracts/            # OpenAPI/AsyncAPI/shared schemas
project-config-schema/ # talos.yaml schema
agent-adapter-spec/   # adapter protocol definitions
```

```
infra/
docker-compose.dev.yml
dokploy/
postgres/
rabbitmq/
redis/
```

```
docs/
architecture.md
security-model.md
agent-run-lifecycle.md
provider-auth.md
deployment.md
```

5.1 Can it be one codebase?

Yes. It should be one codebase, but it should not be one runtime. The agent runner must be isolated from the API and web UI because it runs shell commands, clones repositories, and executes potentially unsafe code. A monorepo with multiple deployables is the right compromise.

6. Core services

6.1 Angular web dashboard

The dashboard is the main product surface. It should expose the following modules:

Module	MVP behavior
Command Center	Shows all projects, running agent jobs, blocked approvals, recent failures, and suggested next actions.
Project Registry	Create/update project records, repo URLs, stack type, build/test commands, deploy targets, and risk rules.
Kanban Board	Backlog, Ready, Running, Review, Blocked, Done.

Module	MVP behavior
	Cards map to tasks and can start agent runs.
Agent Runs	Live stream logs, status timeline, current step, model/agent used, artifacts.
Review Center	Diff viewer, changed files, test results, risk notes, approve/reject/request-changes buttons.
Integrations	Configure GitHub, GitLab, Dokploy, Telegram, WhatsApp, provider credentials and local CLI modes.
Memory/Docs	Project conventions, architecture notes, run history, decision log, stack playbooks.

6.2 Spring Boot control API

The API is the authoritative control plane. It owns persistent state, authentication, permissions, task lifecycle, approval workflow, and external integration metadata.

Core API packages:

- io.talos.auth
- io.talos.projects
- io.talos.tasks
- io.talos.runs
- io.talos.approvals
- io.talos.integrations
- io.talos.notifications
- io.talos.audit
- io.talos.memory
- io.talos.webhooks

6.3 Python orchestrator

The orchestrator coordinates asynchronous work. It should be stateless where practical, with state persisted in PostgreSQL and run events flowing through RabbitMQ. Keep the MVP simple; LangGraph or Temporal can be added later if workflows become complex.

Orchestrator responsibilities:

- Consume TASK_RUN_REQUESTED events.
- Load project config and task metadata.
- Check policy and approval requirements.
- Select agent adapter and worker image.
- Ask runner supervisor to prepare workspace.
- Start the agent session.
- Stream logs and status events.
- Trigger build/test steps.
- Ask reviewer agent or static reviewer to summarize diff.
- Request approval or open PR.

7. Agent adapter layer

The most important abstraction is the `AgentAdapter`. Every agent provider should look the same to your orchestration layer, even if the implementation uses an API call, command-line process, ACP server, MCP bridge, or OpenHands backend.

```
interface AgentAdapter {
    ProviderCapabilities capabilities();
    SessionHandle startSession(AgentSessionRequest request);
    void sendPrompt(SessionHandle session, AgentPrompt prompt);
    Stream<AgentEvent> streamEvents(SessionHandle session);
    AgentRunStatus getStatus(SessionHandle session);
    void stopSession(SessionHandle session);
    AgentResult collectResult(SessionHandle session);
}
```

ProviderCapabilities:

- supportsStreaming
- supportsSubscriptionAuth
- supportsApiKeyAuth
- supportsHeadlessMode
- supportsWorktreeMode
- supportsACP
- supportsMCP
- supportsDiffOutput
- supportsApprovalHooks

Adapter	MVP approach	Notes
OpenCodeAdapter	CLI wrapper or server wrapper	Good first adapter because it is open and provider-flexible.
CodexCliAdapter	CLI wrapper using ChatGPT sign-in for personal use or API key for automation	Use official API-key mode for CI/server automation where possible.
ClaudeCodeAdapter	CLI wrapper for personal/local workflows; API key / SDK mode for productized server workflows	Do not build a SaaS around customer Claude subscription login unless provider-approved.
OpenHandsAdapter	HTTP/API/ACP integration	Use as an optional execution runtime or sandboxed backend.
GeminiCliAdapter	CLI wrapper/ACP where available	Lower priority unless you already use it heavily.
CustomShellAdapter	Runs deterministic scripts only	Useful for build/test/deploy steps that are not agent-driven.

8. Workspace and execution model

Every agent run should execute inside an isolated workspace. The MVP can use git worktrees plus Docker. Later, stronger isolation can be added with rootless Docker, Firecracker, gVisor, or Kubernetes.

Run lifecycle:

1. Receive task.
2. Validate project and permissions.
3. Create run record.
4. Clone repository or fetch latest.
5. Create branch/worktree: agent/<task-id>-<slug>.
6. Mount workspace into selected worker container.
7. Inject only allowed secrets.
8. Start selected agent adapter.
9. Stream logs and events.
10. Capture file changes.
11. Run tests/build.
12. Generate review summary.
13. Create approval request.
14. On approval, create PR or deploy.
15. Clean up old workspace after retention period.

Risk	Required guardrail
Agent deletes important files	All runs happen on task branches/worktrees; deletions require review.
Agent reads secrets	Do not mount .env files by default; use scoped secret injection only when approved.
Agent runs destructive command	Policy engine blocks or approval-gates commands such as sudo, rm -rf, docker system prune, psql writes, systemctl, ssh deploy.
Agent affects host	No host root. Prefer non-root containers and avoid mounting Docker socket into agent containers.
Run consumes VPS resources	Set CPU/memory/time/disk limits per run and cap concurrency.
Provider credentials leak	Keep provider auth in isolated home directories or secret mounts; never expose tokens in logs.

9. Data model

users

id, name, email, password_hash/oauth_subject, role, created_at

projects

id, name, slug, repo_url, default_branch, stack_type, status, created_at

project_configs

id, project_id, config_yaml, parsed_json, version, is_active, created_at

project_environments

id, project_id, name, deploy_provider, dokploy_app_id, base_url, requires_approval

tasks

id, project_id, title, description, source, status, priority, risk_level,
requested_by, assigned_agent_key, created_at, updated_at

agent_runs

id, task_id, project_id, status, agent_key, provider_auth_mode,
branch_name, workspace_id, summary, test_status, review_status,
started_at, completed_at

agent_run_steps

id, run_id, step_type, status, started_at, completed_at, summary

agent_run_logs

id, run_id, step_id, sequence, stream, message, created_at

approvals

id, task_id, run_id, approval_type, requested_action, status,
requested_by, approved_by, approved_at, notes

git_changes

id, run_id, file_path, change_type, additions, deletions, diff_summary

pull_requests

id, run_id, provider, pr_number, url, status, created_at

integrations

id, type, name, config_json, enabled, created_at

integration_credentials

id, integration_id, secret_ref, auth_mode, owner_user_id, created_at

memory_documents

id, project_id, title, source_type, content, created_at

memory_chunks

id, memory_document_id, chunk_text, embedding, metadata_json

audit_events

id, actor_user_id, event_type, entity_type, entity_id, details_json, created_at

10. API surface

Project APIs:

GET /api/v1/projects
POST /api/v1/projects
GET /api/v1/projects/{id}
PUT /api/v1/projects/{id}
POST /api/v1/projects/{id}/sync-config
GET /api/v1/projects/{id}/runs

Task APIs:

GET /api/v1/tasks
POST /api/v1/tasks
GET /api/v1/tasks/{id}
PATCH /api/v1/tasks/{id}

POST /api/v1/tasks/{id}/start-run
 POST /api/v1/tasks/{id}/move

Run APIs:

GET /api/v1/runs/{id}
 GET /api/v1/runs/{id}/logs
 GET /api/v1/runs/{id}/events/stream
 GET /api/v1/runs/{id}/diff
 POST /api/v1/runs/{id}/cancel
 POST /api/v1/runs/{id}/rerun-tests

Approval APIs:

GET /api/v1/approvals
 POST /api/v1/approvals/{id}/approve
 POST /api/v1/approvals/{id}/reject
 POST /api/v1/approvals/{id}/request-changes

Integration APIs:

GET /api/v1/integrations
 POST /api/v1/integrations
 POST /api/v1/integrations/github/test
 POST /api/v1/integrations/dokploy/test
 POST /api/v1/webhooks/telegram
 POST /api/v1/webhooks/whatsapp
 POST /api/v1/webhooks/github

11. Event and queue design

Exchange: agentos.events

Routing keys:

task.created
 task.run.requested
 run.workspace.prepare
 run.workspace.ready
 run.agent.started
 run.agent.event
 run.agent.completed
 run.tests.requested
 run.tests.completed
 run.review.requested
 run.review.completed
 approval.requested
 approval.approved
 approval.rejected
 pr.create.requested
 pr.created
 deploy.requested
 deploy.completed
 deploy.failed

Use event rows in PostgreSQL for durable audit, and RabbitMQ for asynchronous processing. Redis can hold short-lived locks such as one active run per project branch.

12. Security and governance

Security is a core feature. Coding agents execute commands, modify files, and may interact with credentials. The platform must be designed as if agent instructions and repository contents can be malicious.

Control	MVP requirement
Identity and auth	Start with single-user login or admin account. Add OAuth/SSO later.
RBAC	Roles: Owner, Maintainer, Reviewer, Viewer.
Policy engine	Block or approval-gate risky files, commands, tools, deployments, and network actions.
Secrets	Use secret refs. Never store plaintext secrets in normal DB rows. Mask logs.
Runner isolation	Non-root container execution, no direct host filesystem, per-run workspace volume.
Network controls	Start with default network access, then add allow/deny rules per project.
Audit trail	Record task creation, prompt, agent selected, commands, files changed, approvals, deploys.
Human approval	Required for production deploys, database migrations, auth/security code, secrets, destructive shell, and remote device access.

Example policy.yaml:

```

commands:
  blocked:
    - "rm -rf /"
    - "chmod 777"
    - "docker system prune"
    - "curl * | sh"
  approval_required:
    - "sudo"
    - "systemctl"
    - "ssh"
    - "psql"
    - "docker compose down"
    - "npm audit fix --force"

files:
  blocked_patterns:
    - ".env"
    - "*.pem"
    - "id_rsa"
    - "**/secrets/**"
```

```

approval_required_patterns:
- ".github/workflows/**"
- "docker-compose*.yaml"
- "src/main/resources/application-prod*.yaml"
- "**/migrations/**"

```

```

deploy:
  production_requires_approval: true
  staging_auto_deploy_allowed: false

```

13. Subscription vs API-key execution strategy

The platform should support both personal subscription-based CLI use and API-key-based automation, but these must be treated differently.

Mode	Use case	Recommendation
Personal subscription CLI	You run the platform for yourself on your workstation or VPS, using your own authenticated Codex/Claude Code CLI session.	Acceptable for personal workflow. Keep credentials isolated and do not expose this mode to other users.
API key / BYOK	Server-side automation, CI, monetized product, team mode, customer deployments.	Preferred mode for productized workflows. Each provider key is explicitly configured and billed according to provider terms.
Hybrid local executor	A user runs a local agent runner on their own machine that connects back to the web dashboard.	Best future commercial model if users want subscription-based local agents without sending their account tokens to your server.
Hosted multi-tenant subscription proxy	Your server uses customers' Claude/OpenAI subscription login for automated work.	Avoid unless explicitly supported and approved by the provider.

14. Project configuration spec

Every managed repo should optionally include an `talos.yaml` file. The platform can also store an equivalent config in the database for projects where you do not want to commit config into the repo.

```

project:
  name: konvelo-backend
  type: spring-boot
  repo: git@github.com:org/konvelo-backend.git
  default_branch: main

```

```

stack:
  language: java
  runtime: java-21
  framework: spring-boot

```

package_manager: maven

commands:

install: "./mvnw dependency:resolve"

test: "./mvnw test"

build: "./mvnw clean package -DskipTests=false"

lint: null

agents:

preferred: codex-cli

allowed:

- codex-cli
- claude-code
- opencode

rules:

require_approval_for:

- production_deploy
- database_migration
- auth_changes
- security_config
- dependency_major_upgrade

forbidden:

- commit_secrets
- modify_env_files
- force_push_main

context:

docs:

- docs/architecture.md
- docs/api-guidelines.md

important_paths:

- src/main/java
- src/main/resources

ignore_paths:

- target
- node_modules
- .env

deploy:

provider: dokploy

app_id: backend-app-id

environment: production

approval_required: true

15. Dashboard and UX scope

Screen	First version behavior
Command Center	Cards for active runs, approvals waiting, failed builds, and recently completed tasks.
Projects	List projects by portfolio: full-time, business,

Screen	First version behavior
	freelance, personal. Show stack, repo health, latest run, deploy target.
Project Detail	Overview, tasks, runs, config, memory, commands, deployments, repository links.
Kanban	Drag cards between states. Start agent run from a card. Show agent/status badges.
New Task	Natural language description, project picker, priority, agent choice, approval mode.
Run Detail	Timeline, logs, current step, changed files, artifacts, test results.
Review	Diff viewer, comments, risk summary, approve PR, reject, request changes, rerun tests.
Integrations	Git provider, Dokploy, model providers, local CLI mode, webhooks, Telegram/WhatsApp.

16. Implementation phases and tickets

Phase 0 - Foundations

- Create monorepo structure.
- Add Docker Compose for local dev: postgres, rabbitmq, redis, api, web, orchestrator.
- Define OpenAPI contracts for projects, tasks, runs, approvals.
- Add database migrations with Flyway/Liquibase.
- Implement basic auth and single-owner mode.
- Add global audit_event writer.

Phase 1 - Project registry and Kanban

- Build Projects CRUD in Spring Boot.
- Build Tasks CRUD with statuses Backlog, Ready, Running, Review, Blocked, Done.
- Build Angular project list and project detail screens.
- Build Angular Kanban board with drag/drop and task detail drawer.
- Add project stack profiles for Spring Boot, Angular, Astro, Python, Odoo, Flutter.
- Add talos.yaml parser and schema validation.

Phase 2 - Agent run MVP

- Implement RabbitMQ event publishing for task.run.requested.
- Implement Python orchestrator consumer.
- Implement workspace manager: clone repo, create branch/worktree, prepare run directory.
- Implement OpenCodeAdapter or CustomShellAdapter first.
- Implement process log streaming into agent_run_logs.
- Run project-defined build/test command after agent completes.
- Capture git diff and changed-file summary.
- Display run detail screen in Angular.

Phase 3 - Review and PR flow

- Add approval records and Review Center screen.
- Add diff viewer with file tree and unified diff.
- Add PR creation integration for GitHub first.
- Add reject/request-changes actions.
- Add reviewer summary generation using selected model/API.
- Add policy engine for files/commands/actions requiring approval.

Phase 4 - Multi-agent adapters

- Implement CodexCliAdapter.
- Implement ClaudeCodeAdapter.
- Implement OpenHandsAdapter or ACP adapter.
- Add provider capability registry.
- Add provider auth profiles: subscription-local, API-key, BYOK, disabled.
- Add manual agent selection and basic auto-routing by stack/task type.

Phase 5 - Remote triggers and deployment

- Add Telegram bot adapter first because it is easier than WhatsApp.
- Add inbound webhook parser to create tasks from messages.
- Add approval actions from Telegram.
- Add Dokploy deployment trigger integration.
- Add environment-specific deploy approval gates.
- Add WhatsApp intake later using your existing Konvelo/Meta experience.

Phase 6 - Memory, recommendations, and polish

- Add memory_documents and memory_chunks.
- Index project READMEs, docs, talos.yaml, and past successful runs.
- Add project recommendations: stale tasks, failing builds, missing tests.
- Add cost tracking per provider/API key.
- Add workspace cleanup and retention policies.
- Add team/RBAC mode if monetizing.

17. Testing strategy

Layer	Tests
Backend unit tests	Project/task/run state transitions, policy engine, approval rules, config parser.
Backend integration tests	PostgreSQL migrations, RabbitMQ publishing/consuming, GitHub/Dokploy mock integrations.
Frontend tests	Kanban movement, run detail display, approval actions, form validation.
Orchestrator tests	Run lifecycle state machine, adapter mocks, retry behavior, timeout handling.
Runner tests	Workspace creation, branch naming, diff capture,

Layer	Tests
	log streaming, cleanup.
Security tests	Blocked commands, secret masking, file pattern blocking, no write to main branch.
End-to-end smoke tests	Create task -> run dummy adapter -> capture diff -> request approval -> create fake PR.

18. Dokploy deployment plan

Deploy each service separately in Dokploy, backed by persistent volumes for PostgreSQL, RabbitMQ, Redis, and workspace/artifact storage. Start with one VPS and low concurrency.

Dokploy apps/services:

```
agentos-web
agentos-api
agentos-orchestrator
agentos-runner-supervisor
agentos-postgres
agentos-rabbitmq
agentos-redis
agentos-minio      # optional later
```

Initial concurrency:

```
max_active_agent_runs: 1
max_active_test_jobs: 1
max_workspace_age_days: 7
max_run_timeout_minutes: 60
```

Service	Suggested initial resources
web	0.25-0.5 vCPU, 256-512 MB RAM
api	1 vCPU, 1-2 GB RAM
orchestrator	0.5-1 vCPU, 512 MB-1 GB RAM
runner supervisor	1-2 vCPU shared, 1-2 GB RAM plus worker container limits
postgres	1-2 vCPU, 2-4 GB RAM for MVP
rabbitmq	0.5-1 vCPU, 512 MB-1 GB RAM
redis	0.25-0.5 vCPU, 256-512 MB RAM

19. Risks and mitigations

Risk	Mitigation
Provider subscription terms change or block automation	Keep API-key/BYOK path first-class. Add local-runner mode for personal subscription use.
Agent adapters break due to CLI output changes	Wrap adapters behind contract tests and version capability checks. Prefer protocol/API where available.
VPS resource exhaustion	Strict concurrency, container limits, timeouts,

Risk	Mitigation
	cleanup jobs, build worker separation.
Security incident through malicious repo/config	Sandboxing, policy engine, no automatic secret mount, audit logs, per-project trust levels.
Scope creep	MVP supports three stacks and one agent adapter before Telegram/WhatsApp/Odoo/Flutter.
Difficult diff/review UX	Start with unified diff text; add rich diff viewer later.
Monetization conflicts with provider subscriptions	Commercial version should use API keys, BYOK, or user-owned local runners, not shared subscription auth.

20. Claude Code execution prompt

Use the following prompt to have Claude Code scaffold and implement the platform incrementally. It is also provided as a separate Markdown file.

You are Claude Code working as a senior full-stack architect and implementation agent. Build the first working version of Talos: a self-hosted, web-based, agent-agnostic control plane for managing coding tasks across many repositories.

Important product definition:

- Do not build a new coding agent.
- Build the harness/control plane that can orchestrate existing agents through adapters.
- Start with a simple CustomShellAdapter and OpenCodeAdapter placeholder. Add CodexCliAdapter and ClaudeCodeAdapter extension points, but do not require them to work in the first commit.
- Use a monorepo with multiple deployable services.

Canonical naming conventions:

- Product/UI name: Talos.
- Repository/root folder: talos/.
- Java base package: io.talos.
- Docker service names: talos-api, talos-web, talos-orchestrator, talos-runner-supervisor.
- Environment variable prefix: TALOS_.
- Database name: talos.
- Queue/exchange prefix: talos.events and talos.task.run.requested.
- Project config file: talos.yaml. Optional legacy aliases may be added later, but do not make them the MVP default.
- Do not use previous working-title names in the generated codebase.

Tech stack:

- Frontend: Angular.
- Backend API: Spring Boot, Java 21.
- Database: PostgreSQL.
- Queue: RabbitMQ.
- Cache: Redis.
- Orchestrator: Python.
- Runner model: git worktree or cloned workspace per run, later Dockerized workers.
- Deployment target: Docker/Dokploy.

Create this repo structure:

talos/
 apps/web
 apps/api
 apps/orchestrator
 apps/runner-supervisor
 workers/base-agent-runner
 workers/java-runner
 workers/node-runner
 workers/python-runner
 packages/contracts
 packages/project-config-schema
 infra/docker-compose.dev.yml
 docs/

MVP requirements:

1. Spring Boot API

- Implement entities/tables for users, projects, project_configs, tasks, agent_runs, agent_run_steps, agent_run_logs, approvals, git_changes, pull_requests, integrations, audit_events.
- Use migrations.
- Implement REST APIs for projects, tasks, runs, approvals, and integrations.
- Implement task statuses: BACKLOG, READY, RUNNING, REVIEW, BLOCKED, DONE, CANCELLED.
- Implement run statuses: CREATED, QUEUED, PREPARING_WORKSPACE, RUNNING_AGENT, RUNNING_TESTS, REVIEWING, WAITING_APPROVAL, COMPLETED, FAILED, CANCELLED.
- Implement event publishing to RabbitMQ when a task run is requested.
- Implement policy checks for blocked file patterns and risky commands.

2. Angular web app

- Build a dashboard shell with navigation.
- Build Projects list/create/edit screens.
- Build Project detail screen.
- Build Kanban board for tasks.
- Build New Task form.
- Build Run detail screen with logs and status timeline.
- Build Review Center screen showing changed files, diff text, test result, approve/reject buttons.

3. Python orchestrator

- Consume task.run.requested messages from RabbitMQ.
- Load project and task metadata from API or database.
- Create agent_run record/state updates through API.
- Ask runner-supervisor to prepare workspace and run adapter.
- Capture logs and update run status.
- Run project test/build commands from talos.yaml.
- Capture git diff and changed file summary.
- Create approval request when run completes.

4. Runner supervisor

- Implement local development mode first.
- Clone or reuse repo workspace.
- Create branch/worktree named agent/task-`<id>-<slug>`.
- Run a dummy CustomShellAdapter that can execute a configured script or create a test file for smoke testing.
- Stream stdout/stderr back to orchestrator.
- Capture git status and diff.

5. Project config

- Define talos.yaml schema.
- Parse project name, type, repo, branch, commands, agents, approval rules, context docs, deploy provider.
- Store parsed config in project_configs.

6. Local dev infrastructure

- Provide docker-compose.dev.yml for PostgreSQL, RabbitMQ, Redis, API, web, orchestrator, and runner-supervisor.
- Include README.md with setup instructions.
- Include .env.example files.

7. Tests

- Add backend unit tests for state transitions, policy engine, config parser.
- Add orchestrator tests with mocked adapter.
- Add an end-to-end smoke test path: create project -> create task -> start run -> dummy adapter runs -> diff captured -> approval requested.

Implementation style:

- Keep service boundaries clean.
- Do not hardcode secrets.
- Do not store plaintext provider keys in normal database fields.
- Do not implement production deployment in the first pass; stub Dokploy integration with an interface.
- Use clear TODO comments for CodexCliAdapter, ClaudeCodeAdapter, OpenCodeAdapter, OpenHandsAdapter.
- Make the first version runnable locally even if no real coding agent is configured.

Deliverables:

- Working monorepo scaffold.
- API compiles and tests pass.
- Angular app starts and can create/list projects and tasks.
- Orchestrator consumes run requests.
- Runner supervisor can execute a dummy run and produce logs/diff.
- README with exact commands to run the system locally.

Work incrementally:

1. Scaffold repo and dev infra.
2. Build backend models/APIs/migrations.
3. Build frontend basic screens.
4. Build orchestrator and runner dummy flow.
5. Connect end-to-end run lifecycle.
6. Add tests and README.
7. Stop and summarize what works, what is stubbed, and the next implementation steps.

21. References

- OpenHands homepage and positioning: <https://www.openhands.dev/>
- OpenHands Docker sandbox guide: <https://docs.openhands.dev/sdk/guides/agent-server/docker-sandbox>
- OpenHands Agent Canvas repository: <https://github.com/OpenHands/agent-canvas>
- OpenHands ACP agents documentation:
<https://docs.openhands.dev/openhands/usage/agent-canvas/acp-agents>

- Vibe Kanban GitHub repository: <https://github.com/BloopAI/vibe-kanban>
- Vibe Kanban documentation: <https://vibe-kb.com/docs/getting-started/>
- Nimbalyst GitHub repository: <https://github.com/nimbalyst/nimbalyst>
- OpenClaw ACP agents documentation: <https://docs.openclaw.ai/tools/acp-agents>
- OpenAI Codex authentication documentation: <https://developers.openai.com/codex/auth>
- Claude Code Agent SDK overview and third-party auth note: <https://code.claude.com/docs/en/agent-sdk/overview>
- Claude Code authentication documentation: <https://code.claude.com/docs/en/authentication>
- BuilderMethods Agent OS: <https://buildermethods.com/agent-os>